

Steps of the stage 1

Responsible for the stage: Lydia

Tasks complete: preprocessing, stage 1

Overall time: 398 sec (approximately)

Preprocessing: 13 features extracted from the original dataset.

1. `nr\_passag\_pagos`: Number of passengers who occupy seats and generate revenue.
2. `nr\_passag\_gratis`: Number of passengers who occupy seats marketed to the public, but do not generate revenue.
3. `ds\_servico\_tipo\_linha`: Description of the service referring to the type of line (Passenger/Cargo).
4. `kg\_carga\_paga`: Cargo paid. It is the total amount, expressed in kilograms, of all goods that have been transported on the aircraft, except mail and luggage, and have generated direct or indirect revenue for the airline.
5. `id\_aerodromo\_origem`: Identifier code of the airfield of origin in the system.
6. `nm\_pais\_origem`: Name of the country of origin.
7. `id\_aerodromo\_destino`: Identification code of the destination aerodrome in the system.
8. `nm\_pais\_destino`: Name of the country of destination.
9. `ds\_grupo\_di`: Description of the DI group or Flight Type. Refers to the grouping of DI classifications into types of operations:
 - Regular
 - Non-regular
 - Unproductive
10. `ds\_natureza\_tipo\_linha`: Description of the nature referring to the type of line (International/Domestic).
11. `ds\_tipo\_empresa`: Description of the type of company. It refers to the description of the type of company in relation to the service performed. It may contain the following descriptions:
 - Regular air transport
 - Non-regular air transport
 - Regular domestic
 - Regular foreigner
 - Non-regular foreigner
 - Public air services
 - Regular systematic airway connection
 - Air taxi
 - Aerial
 - Aeropublicity
 - Aerophotography
 - Air inspection
12. `nr\_ano\_partida\_real`: Number of the actual departure year.
13. `nr\_mes\_partida\_real`: Number of the real departure month.

Stage 1: the selected columns are combined into a single flight facts table. After the schema is created and the data is loaded, the entire database is migrated to HDFS.

Format	Codec	Write Time (s)	Read Time (s)	Size (bytes)	Size (MB)
text	gzip	232	21.46	459912597	438.6
text	snappy	158	17.62	841503320	802.5

avro	snappy	406	21.0	919838007	877.2
avro	deflate	458	19.23	535518939	510.7
parquet	snappy	322	7.66	430738460	410.8
parquet	gzip	358	7.61	294851536	281.2

Parquet + gzip were used due to most sufficient storage size and read time.

Steps of the stage 2

Responsible for the stage: Ivan and Lydia

Tasks complete: stage 2 (table optimization, analytical queries, hive partitions)

Overall time: 262 sec (approximately)

Table optimization (comprehensive pipeline):

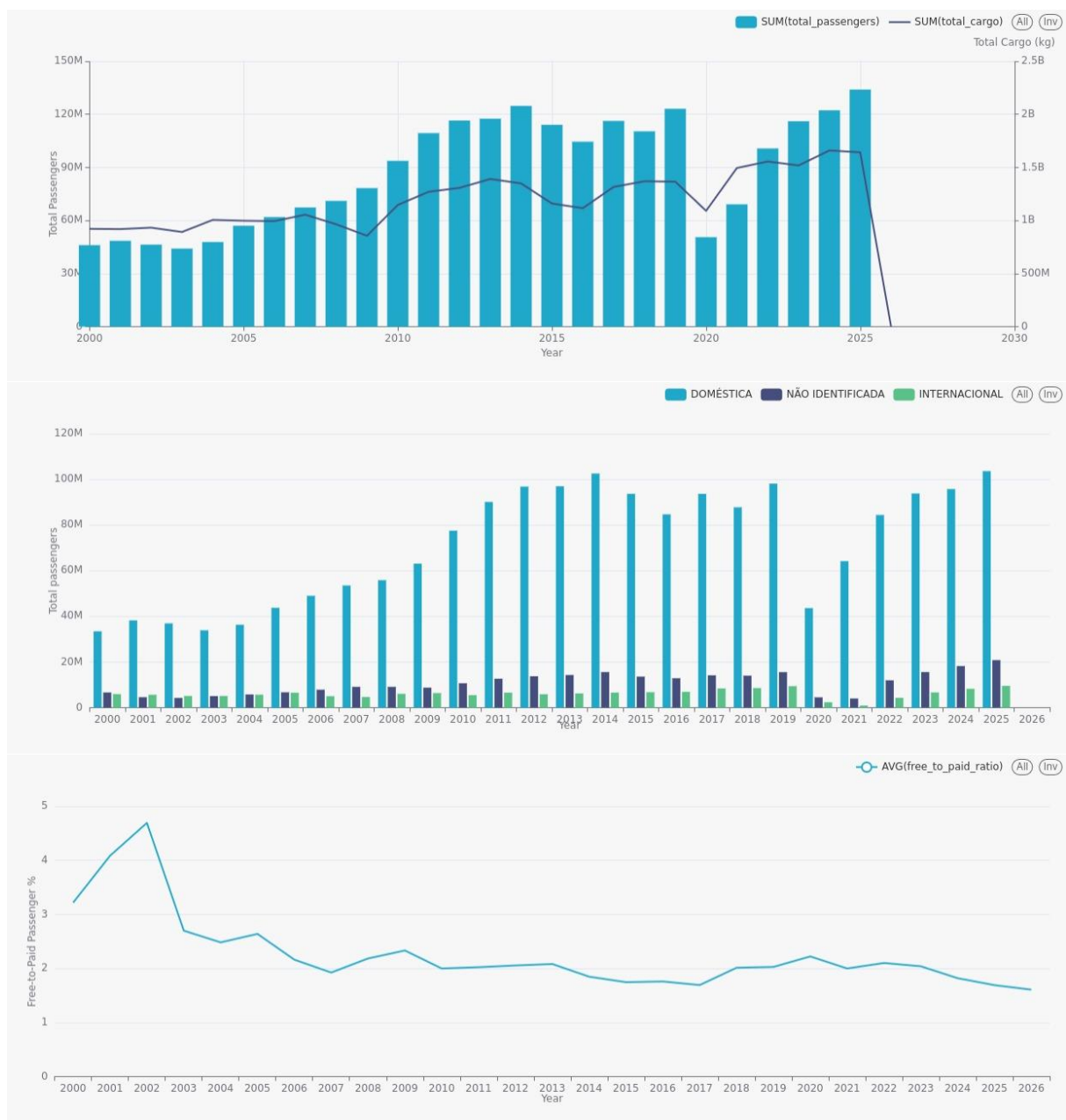
1. Removes the database if it already exists (CASCADE drops all contained tables). Creates a fresh database with a specified HDFS location (project/hive/warehouse relative to the user's home). Selects the database so subsequent statements operate inside it.
2. Dynamic partitioning is enabled and set to nonstrict mode, so that all partitions can be created automatically during data insertion (no need for a static partition prefix). Bucketing enforcement ensures that when data is inserted into a bucketed table, the number of reducers will match the number of buckets. Parquet compression is set to GZIP, so data stored in Parquet format will be compressed with Gzip (consistent with the decision from Stage 1).
3. Drops the table if it exists (idempotent). Creates an external table that reads the Parquet files already stored in HDFS by Sqoop (at project/warehouse/fact_flight). The schema matches the original PostgreSQL table. This table does not own the data; dropping it will only remove the metadata.
4. Creates a managed table (no EXTERNAL keyword), so Hive will manage its lifecycle and store the data inside the Hive warehouse directory (project/hive/warehouse/fact_flights_optimized). The column nr_ano_partida_real is removed from the column list and used as the partition key instead. The table is bucketed on ds_natureza_tipo_linha with 8 buckets, which improves join and sampling performance when filtering on that column. The storage format remains Parquet (compression is Gzip due to the earlier SET).
5. Dynamically inserts all rows from the raw external table into the optimized table. Because PARTITION (nr_ano_partida_real) is specified with no static value, Hive automatically creates a separate partition for each distinct year found in the data. The SELECT lists all columns except nr_ano_partida_real on purpose, and then includes nr_ano_partida_real as the last column in the select list – this is the required order for dynamic partition inserts (the partition column must come last).
6. SHOW PARTITIONS lists all the year partitions that were created (e.g., nr_ano_partida_real=2000, 2001, ...). The SELECT query counts the number of flights per year, confirming the data was correctly distributed among partitions.
7. After the data has been successfully moved into the optimized managed table, the raw external table is dropped. The underlying HDFS files (the Parquet data from Sqoop) remain untouched because the table was EXTERNAL, but the Hive metadata is deleted. From now on, all EDA and ML stages should use only fact_flights_optimized

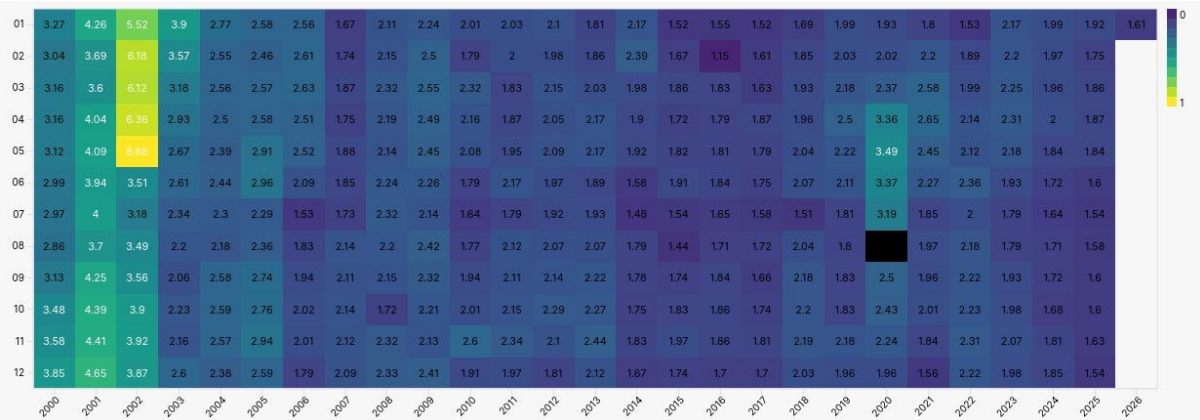
Queueries:

1. q1: Identifies monthly demand patterns, peak travel months, and seasonal fluctuations over multiple years.
2. q2: Provides a high-level annual trend of air travel demand, useful for long-term strategic planning and growth analysis.
3. q3: Detects operational inefficiencies (e.g., too many free seats on certain routes/months) and helps airlines optimize seat allocation and loyalty/free ticket programs.

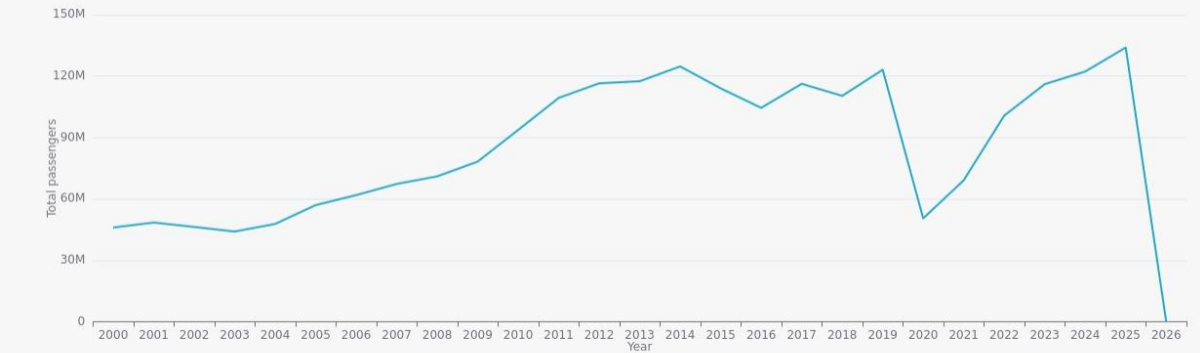
4. q4: Reveals year-over-year trends in airline seat giveaway practices, useful for evaluating changes in marketing, loyalty programs, or regulatory requirements.
5. q5: Helps compare domestic vs. international demand growth, identify shifting travel preferences, and support decisions on fleet allocation and route expansion.
6. q6: Reveals dual-revenue stream patterns (passenger + cargo). A year with stable passengers but surging cargo may indicate e-commerce growth or freight market shifts. Also helps detect seasonal correlation between passenger and cargo demand.

Information about hive tables is presented in the `output/hive\_results.txt` file.





SUM(total_passengers) (All) (Inv)



Steps of the stage 3

Responsible for the stage: Polina

Tasks complete: stage 3 (SparkML pipeline)

Overall time: 5 hours +

ML pipeline:

1. Filtering and cleaning: raw flight data is filtered to retain only regular passenger flights with complete year, month, and passenger information. Outliers are removed (paid passengers between 1 and 853, total passengers ≤ 900) and records up to 2025 are kept, giving a clean dataset where each row represents one flight and the target is total passengers per month.
2. Monthly aggregation: flight-level records are grouped by date to sum total passengers per month. This converts granular flight data into a structured monthly time series with each row representing one month and the label being the total passenger volume for that month.
3. Calendar completion: a complete monthly timeline is generated between the earliest and latest dates, then left-joined with the actual aggregated data. Missing months are filled with zero passengers and an `is_filled` flag is added to indicate whether the observation was artificially inserted.
4. Basic time features: from each date, year, month, and quarter are extracted. A linear trend variable `time_index` is computed as $(\text{year} - \text{min_year}) * 12 + \text{month}$ to encode the progression of time across the series.
5. Log transformation of target: the target variable is transformed using $\log(1 + x)$ to stabilize variance and reduce the influence of large peaks. All models are trained on the log-scale target, and predictions are later converted back using $\exp(x)$ to the original passenger scale.
6. Lag features: window functions generate lagged values of the log-transformed passenger counts: 1 month (previous month), 12 months (same month last year), and 24 months (two years prior). These features capture short- and long-term autocorrelation and seasonality patterns.
7. Rolling statistics: rolling means over 3, 12, and 24-month windows summarize recent, yearly, and long-term trends. The rolling standard deviation over 12 months captures yearly variability. Together they provide a smoothed view of historical behaviour rather than relying on single-point lags.
8. Cyclical encoding of month: the month integer is transformed into sine and cosine components using a 12-month cycle. This preserves the natural circular relationship (December and January are neighbours) and avoids the false linear distance a numerical 1–12 encoding would imply. The pipeline transforms historical flight records into a monthly time-series dataset for supervised learning. With lagged values, rolling statistics, time features, and cyclical encoding, the models can forecast future monthly passenger volumes by learning seasonal patterns and long-term trends.
9. Data split: train 0.75/ test 0.25
10. Time-series cross-validation: the training set grows over time, and each fold validates on the subsequent time period. This preserves temporal order and prevents data leakage. Hyperparameter tuning is performed using this custom time-aware validation strategy.
11. Two models are trained and compared: SparkXGBRegressor and GBRegressor.

Results

GBRegressor Test Results



SparkXGBRegressor Test Results

